A D PATEL INSTITUTE OF TECHNOLOGY
THE CHARUTAR VIDYA MANDAL
DEPT. OF INFORMATION TECHNOLOGY

HOW NETFLIX RECOMMENDS MOVIES

The Role of Algorithms

A Design and Analysis of Algorithms (DAA) Project Report

An educational study of recommendation techniques with a C++ demonstration

A report submitted
In partial fulfilment of the requirements for the Degree of
Bachelor of Technology
In
Information Technology
Semester – V

Subject:
Design Analysis of Algorithm (202045601)

By
PARMAR DWIJA (12502080603005)

Faculty Guide
Minakshi Patil
Assistant Professor
Academic Year 2026

Department of Information Technology
A D Patel Institute of Technology
New Vallabh Vidyanagar, Vitthal Udyognagar – 388121
Anand (Gujarat), India

A D PATEL INSTITUTE OF TECHNOLOGY
THE CHARUTAR VIDYA MANDAL
DEPT. OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project work embodied in this report entitled “How Netflix Recommends Movies – The Role of Algorithms” was prepared by Parmar Dwija (Enrollment No.: 12502080603005) at A D Patel Institute of Technology for partial fulfilment of the requirements of the Bachelor of Technology degree in Information Technology.

The report studies recommendation algorithms, including collaborative filtering, content-based filtering, ranking and hybrid approaches. The C++ program included in this report is an educational simulation created to demonstrate recommendation logic; it is not Netflix's proprietary production system.

Date: _____

Place: New Vallabh Vidyanagar

DECLARATION

I, Parmar Dwija (Enrollment No.: 12502080603005), declare that this report entitled "How Netflix Recommends Movies – The Role of Algorithms" has been prepared by me for the subject Design and Analysis of Algorithms (202045601) in Semester V of the Bachelor of Technology programme in Information Technology.

The report explains recommendation concepts and presents an educational C++ implementation for ranking movies from simple user ratings and genre preferences. The implementation is my learning model and is not claimed to reproduce Netflix's private algorithms or internal data.

Where definitions and general technical concepts are drawn from published material, they are acknowledged in the References section.

Date: _____ PARMAR DWIJA

Place: New Vallabh Vidyanagar (12502080603005)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my subject teacher and department for giving me the opportunity to study recommendation algorithms through this Design and Analysis of Algorithms project.

This project helped me connect algorithmic ideas with a familiar streaming service. I learned how user ratings, movie genres, similarity measures and ranking rules cmaruan be combined to create a useful list of movie suggestions.

I am thankful to everyone who supported my learning of C++ programming, data structures, sorting, scoring methods and complexity analysis. I also acknowledge the official Netflix Technology Blog and other educational references that help explain recommendation systems at a high level.

PARMAR DWIJA
(12502080603005)

TABLE OF CONTENTS

Section	Page (approx.)
Abstract	6
Chapter 1: Introduction	7
Chapter 2: Problem Statement and Objectives	8
Chapter 3: Theoretical Background and Recommendation Algorithms	9
Chapter 4: Data Design and Movie Model	11
Chapter 5: Algorithm I — Content-Based Scoring	12
Chapter 6: Algorithm II — Collaborative Filtering and Ranking	13
Chapter 7: System Design and C++ Implementation	15
Chapter 8: Testing and Comparative Results	17
Chapter 9: Sample Execution and Evidence	19
Chapter 10: Complexity Analysis and Correctness	21
Chapter 11: Limitations, Future Scope and Conclusion	22
References and Appendix	23

ABSTRACT

Netflix offers a large catalogue of films and series, so a viewer cannot manually inspect every title before choosing what to watch. Recommendation algorithms help organise this catalogue and estimate which titles may be relevant to a particular member. These systems can use viewing history, ratings or other interactions, information about titles, and patterns shared by users with similar interests.

This report explains the role of algorithms in movie recommendation at a high level. It introduces content-based filtering, collaborative filtering, hybrid recommendation and ranking. It then presents an educational C++ program that scores a small movie catalogue using genre similarity and sample user preferences, sorts the candidates, and displays the highest-scoring suggestions.

The demonstration uses invented sample data so that every calculation can be checked. It is not Netflix's actual production algorithm, which is complex, changes over time and is not fully public. The report also discusses testing, correctness, time and space complexity, limitations and possible improvements.

Keywords: recommendation system, Netflix, algorithms, content-based filtering, collaborative filtering, ranking, C++.

CHAPTER 1: INTRODUCTION

1.1 Background

Streaming platforms provide users with many movies and series. A large catalogue gives viewers more choice, but it can also make selection difficult. A recommendation system helps reduce this effort by presenting titles that are likely to match a viewer's interests.

Netflix has publicly described using recommendation technology to personalise the experience. At a general level, recommendation systems analyse signals such as viewing activity and information about titles, then rank possible choices. The exact production system is proprietary and should not be assumed to be identical to the simplified model in this report.

1.2 Role of Algorithms

An algorithm is a finite sequence of steps used to solve a problem. In a recommendation system, algorithms can collect useful signals, calculate similarity or predicted interest, rank candidate titles, and select which recommendations to display.

- Personalisation: different viewers can receive different suggestions.
- Filtering: titles that are less relevant can be placed lower in the list.
- Ranking: candidate titles can be ordered by an estimated relevance score.
- Learning from feedback: viewing and interaction patterns can help update future recommendations.

1.3 Why This Topic Matters

Recommendation systems are used in streaming, shopping, music, news and social platforms. Studying them demonstrates how data structures, similarity calculations, sorting and evaluation can solve a practical information-retrieval problem.

1.4 Project Overview

This project uses a small, fixed movie dataset. Each movie has a title and genre tags. The sample viewer has genre preferences. The program calculates a simple match score, sorts the movies by score and prints a ranked list. A separate explanation describes how collaborative filtering can add information from ratings by other users.

1.5 Report Organisation

The next chapters define the problem, explain recommendation methods, describe the data model and algorithms, present C++ code and test cases, and analyse performance and limitations.

CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Given a catalogue of movies and a viewer profile, produce an ordered list of movies that may interest the viewer. The educational program represents a viewer's preferred genres and each movie's genre tags. It calculates a score for each movie and sorts the catalogue from highest score to lowest score.

2.2 Objectives

- Explain how recommendation algorithms help users discover movies.
- Describe content-based filtering, collaborative filtering and hybrid approaches.
- Represent movies and viewer preferences using suitable C++ data structures.
- Calculate a simple, transparent recommendation score.
- Sort candidate movies by score and display the top recommendations.
- Test normal cases, ties, empty preferences and unknown genre matches.
- Analyse the time and space complexity of the demonstration.

2.3 Scope

The program is a console-based educational prototype. It uses a small, hard-coded catalogue and a manually defined preference profile. It does not connect to Netflix, access real viewing histories, call an online service or train a machine-learning model.

2.4 Success Criteria

- Every catalogue item receives a score using the same stated rule.
- Movies with more matching preferred genres receive higher scores.
- The output is sorted from higher score to lower score.
- Equal-score movies are handled consistently.
- The program runs with a standard C++ compiler.

2.5 Assumptions

Genre tags are treated as equally important in this simple model. The sample data is illustrative, and scores represent only the defined genre match—not a real probability that a person will watch a movie.

CHAPTER 3: THEORETICAL BACKGROUND AND RECOMMENDATION ALGORITHMS

3.1 What Is a Recommendation System?

A recommendation system is software that estimates which items may be useful or interesting to a particular user. It can use information about items, user behaviour, ratings and patterns among users. The output is often a ranked list rather than a simple yes-or-no answer.

3.2 Content-Based Filtering

Content-based filtering recommends items that share characteristics with items a user likes. For movies, characteristics may include genres, cast, language, themes, director or descriptive metadata. A viewer who often watches science-fiction titles may receive more science-fiction suggestions.

In this report's C++ example, each movie has genre tags. The score is the number of tags that match the viewer's preferred genres. This is easy to understand, but a real content-based system may use weighted features and more advanced representations.

3.3 Collaborative Filtering

Collaborative filtering uses patterns in interactions from many users. One approach finds users with similar rating behaviour; another finds items that tend to be liked by similar groups of users. If viewers with a similar history enjoyed a particular film, that film may be a useful candidate for another viewer.

Collaborative filtering does not require every movie to be described in detail, but it can struggle with new users or new titles that have little interaction data. This is commonly called the cold-start problem.

3.4 Hybrid Recommendation

A hybrid system combines multiple sources of evidence. For example, it may use title features, viewing patterns, popularity and contextual signals. Combining methods can reduce weaknesses found in any single method. The weight given to each signal depends on the system and its goals.

3.5 Ranking and Scoring

Recommendation systems may first create a set of candidate titles and then rank those candidates. A score can represent estimated relevance. The score is not necessarily a probability and does not guarantee that the viewer will enjoy the title.

3.6 Greedy and Sorting Concepts

A ranked-list prototype can use a sorting algorithm to order movies by score. Sorting does not create the scores; it only arranges the candidates after the scores have been calculated. A greedy selection step can then take the first k movies from the ranked list.

3.7 Privacy and Responsible Use

Personalisation may use sensitive behavioural information. A real service must consider data protection, security, user controls, fairness and transparency.

CHAPTER 4: DATA DESIGN AND MOVIE MODEL

4.1 Movie Structure

The C++ program defines a Movie structure. Each record stores the movie title, its genre tags and the score calculated for the current viewer.

Field	Type	Purpose
title	string	Movie title shown in the output
genres	vector<string>	Genre labels associated with the movie
score	int	Number of genres matching the viewer's preferences

4.2 Viewer Preference Data

The viewer profile is represented by a vector of preferred genre names. The sample viewer prefers Action, Sci-Fi and Adventure. In a real system, preferences might be inferred from repeated interactions instead of being entered manually.

4.3 Sample Catalogue

Movie	Genre tags
Galactic Mission	Sci-Fi, Action, Adventure
The Last Detective	Crime, Drama, Mystery
Jungle Quest	Adventure, Action
Future City	Sci-Fi, Drama
Laugh Out Loud	Comedy, Family
Shadow Agent	Action, Thriller
Ocean Planet	Adventure, Sci-Fi
Family Weekend	Comedy, Family, Drama

4.4 Data Structures

A vector stores the catalogue and another vector stores the preferred genres. Strings hold titles and tags. The program loops through the catalogue to calculate scores and uses sort() with a comparison function to order the results.

4.5 Data Quality

Genre labels must be consistent. For example, “Sci-Fi” and “Science Fiction” would be treated as different strings unless the program normalised them. A larger system should validate missing fields, duplicate titles and inconsistent tags.

CHAPTER 5: ALGORITHM I — CONTENT-BASED SCORING

5.1 Principle

The content-based demonstration compares each movie's genre tags with the viewer's preferred genres. Every matching tag adds one point. A movie with no matching genre receives a score of zero.

5.2 Steps

1. Create a list of the viewer's preferred genres.
2. Read each movie and its genre tags from the catalogue.
3. Set the movie's score to zero.
4. Compare every genre tag with the preferred genre list.
5. Increase the score for each match.
6. Store the score with the movie record.
7. Pass the scored catalogue to the ranking algorithm.

5.3 Pseudocode

```
FOR each movie in catalogue:
    movie.score = 0
    FOR each genre in movie.genres:
        IF genre exists in viewerPreferences:
            movie.score = movie.score + 1
SORT catalogue by score in descending order
DISPLAY the ranked catalogue
```

5.4 Example Calculation

Movie	Matching preferred genres	Score
Galactic Mission	Sci-Fi, Action, Adventure	3
Jungle Quest	Action, Adventure	2
Future City	Sci-Fi	1
Laugh Out Loud	None	0

5.5 Interpretation

The score is transparent and can be checked by hand. It measures only overlap with the selected genre list. It does not consider acting, reviews, release date, language, runtime, a viewer's mood or other possible signals.

CHAPTER 6: ALGORITHM II — COLLABORATIVE FILTERING AND RANKING

6.1 Collaborative Filtering Concept

Imagine that several viewers rate movies from 1 to 5. If two viewers give similar ratings to many of the same titles, their preferences may be similar. A system can use this similarity to identify movies liked by one viewer but not yet seen by the other.

6.2 Simple Similarity Idea

One simple similarity method compares ratings for titles rated by both users. It may calculate how close the ratings are, or use a mathematical similarity measure. More advanced methods can work with large, sparse user-item interaction matrices.

6.3 Why It Is Not Implemented as a Full Model Here

A meaningful collaborative-filtering system needs enough user-item interaction data and a carefully defined rating or implicit-feedback model. This report focuses its working C++ demonstration on content-based genre scoring so that the output is reproducible and easy to verify. Collaborative filtering is explained conceptually rather than presented as a trained production model.

6.4 Ranking Algorithm

After scores are calculated, the program sorts the movies in descending score order. If two movies have the same score, the title is used as a secondary key so the output remains predictable.

```
COMPARE(a, b):
    IF a.score != b.score:
        return a.score > b.score
    return a.title < b.title
```

6.5 Top-K Selection

If the user asks for only three suggestions, the program displays the first three entries after sorting. This is called top-k selection. The chosen k should not be greater than the number of available movies.

6.6 Hybrid Future Version

A future version could calculate a content score and a collaborative score, scale them to comparable ranges, combine them with chosen weights, and rank the combined score. The weights would need evaluation on representative data rather than being selected without evidence.

CHAPTER 7: SYSTEM DESIGN AND C++ IMPLEMENTATION

7.1 Modular Structure

Function / Component	Purpose
Movie structure	Stores title, genres and calculated score
hasGenre()	Checks whether a preferred genre exists in the preference list
calculateScores()	Calculates each movie's content-based score
compareMovies()	Sorts higher scores first and breaks ties by title
main()	Creates sample data, runs scoring and prints recommendations

7.2 Processing Flow

8. Start the program.
9. Create the movie catalogue and viewer preferences.
10. Calculate a match score for every movie.
11. Sort movies by score and title.
12. Display the ranked list and scores.
13. End the program.

7.3 C++ Source Code

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;

struct Movie {
    string title;
    vector<string> genres;
    int score = 0;
};

bool hasGenre(const vector<string>& preferences,
              const string& genre) {
    return find(preferences.begin(), preferences.end(), genre)
           != preferences.end();
}

void calculateScores(vector<Movie>& movies,
                     const vector<string>& preferences) {
    for (auto& movie : movies) {
```

```
            movie.score = 0;
            for (const string& genre : movie.genres) {
                if (hasGenre(preferences, genre)) {
                    movie.score++;
                }
            }
        }
    }

    bool compareMovies(const Movie& a, const Movie& b) {
        if (a.score != b.score)
            return a.score > b.score;
        return a.title < b.title;
    }

    int main() {
        vector<string> preferences = {
            "Action", "Sci-Fi", "Adventure"
        };

        vector<Movie> movies = {
            {"Galactic Mission", {"Sci-Fi", "Action", "Adventure"}},
            {"The Last Detective", {"Crime", "Drama", "Mystery"}},
            {"Jungle Quest", {"Adventure", "Action"}},
            {"Future City", {"Sci-Fi", "Drama"}},
            {"Laugh Out Loud", {"Comedy", "Family"}},
            {"Shadow Agent", {"Action", "Thriller"}},
            {"Ocean Planet", {"Adventure", "Sci-Fi"}},
            {"Family Weekend", {"Comedy", "Family", "Drama"}}
        };

        calculateScores(movies, preferences);
        sort(movies.begin(), movies.end(), compareMovies);

        cout << "NETFLIX-STYLE MOVIE RECOMMENDATION DEMO\n";
        cout << "---------------------------------------\n";
        cout << "Preferred genres: Action, Sci-Fi, Adventure\n\n";
        cout << "Recommended movies (highest match first):\n";

        int rank = 1;
        for (const auto& movie : movies) {
            cout << rank << ". " << movie.title
                 << " | Match score: " << movie.score << '\n';
            rank++;
        }
```

```
    cout << "\nNote: This is an educational simulation, "
            "not Netflix's actual algorithm.\n";
    return 0;
}
```

7.4 Compilation and Execution

```
g++ netflix_recommendation.cpp -o netflix_recommendation
./netflix_recommendation
```

7.5 Error Handling and Improvements

The sample uses fixed, valid data, so it has little input validation. An interactive version should validate user input, handle an empty catalogue, accept multi-word titles safely and check the requested top-k value.

CHAPTER 8: TESTING AND COMPARATIVE RESULTS

8.1 Purpose of Testing

Testing checks that genre matches are counted correctly, scores are sorted in descending order, ties are handled consistently and titles without matching genres still appear at the end of the complete ranked list.

8.2 Test Cases

Test	Input / Situation	Expected behaviour
T1	Movie has three preferred genres	Score is 3
T2	Movie has two preferred genres	Score is 2
T3	Movie has no preferred genres	Score is 0
T4	Two movies have the same score	Titles are ordered alphabetically
T5	All movies are scored	List is sorted highest to lowest
T6	Catalogue contains no movies	Program should display an empty result in an extended version

8.3 Expected Ranked Output

Rank	Movie	Match score
1	Galactic Mission	3
2	Jungle Quest	2
3	Ocean Planet	2
4	Shadow Agent	1
5	Future City	1
6	The Last Detective	0
7	Family Weekend	0
8	Laugh Out Loud	0

8.4 Observations

- Galactic Mission receives the highest score because all three genres match.
- Jungle Quest and Ocean Planet each match two preferred genres.
- Movies with no matching preferred genre receive a score of zero.

8.5 Comparative Discussion

Method	Main signal	Strength	Limitation

Content-based	Movie features and user preferences	Works with item metadata	May over-recommend similar content
Collaborative filtering	Patterns across users or items	Can discover less obvious interests	Needs interaction data; cold-start issues
Hybrid	Combination of signals	Can balance weaknesses	More complex to build and evaluate
This C++ demo	Genre overlap only	Simple and explainable	Not personalised from real viewing data

CHAPTER 9: SAMPLE EXECUTION AND EVIDENCE

9.1 Sample Console Output

```
NETFLIX-STYLE MOVIE RECOMMENDATION DEMO
---------------------------------------
Preferred genres: Action, Sci-Fi, Adventure
```

```
Recommended movies (highest match first):
1. Galactic Mission | Match score: 3
2. Jungle Quest | Match score: 2
3. Ocean Planet | Match score: 2
4. Future City | Match score: 1
5. Shadow Agent | Match score: 1
6. Family Weekend | Match score: 0
7. Laugh Out Loud | Match score: 0
8. The Last Detective | Match score: 0
```

9.2 How to Run

14. Save the program as netflix_recommendation.cpp.
15. Open a terminal in the folder containing the file.
16. Compile using the command shown in Chapter 7.
17. Run the executable and compare the output with the expected ranking.
18. Capture a screenshot of your own run if evidence is required for submission.

9.3 Interpreting the Output

The output displays the preference profile, the ranked movie titles and their scores. The first movie is ranked highest because it matches all three selected genres. Several movies can have the same score; the comparator uses alphabetical title order to make their relative order consistent.

9.4 Evidence to Add

Insert a screenshot of the actual program running on your computer here if your instructor requires execution evidence. Do not label a typed sample as a screenshot of a real program run.

CHAPTER 10: COMPLEXITY ANALYSIS AND CORRECTNESS

10.1 Time Complexity of Scoring

Let n be the number of movies, g the maximum number of genre tags per movie, and p the number of preferred genres. The helper function uses a linear search through preferences for each genre tag. Therefore, score calculation takes O(n × g × p) time in this implementation.

10.2 Time Complexity of Sorting

The standard sort() algorithm used by C++ sorts the n movie records in O(n log n) time in typical standard-library guarantees for comparison sorting. Each comparison takes constant time for the score check, with a title comparison used for ties.

10.3 Overall Time Complexity

The total time is O(n × g × p + n log n). If the number of genre tags and preferences is small and bounded, the scoring stage is approximately O(n), and sorting becomes the main growth term O(n log n).

10.4 Space Complexity

The catalogue stores n movie records and their genre strings, while the preference vector stores p strings. Apart from the input data, sorting may use implementation-dependent auxiliary memory. The stored data space is O(n × g + p), treating each tag as a constant-size item.

10.5 Correctness of Score Calculation

For each movie, the program resets the score to zero and visits every genre tag exactly once. Whenever a tag appears in the preference list, one point is added. Consequently, the final score equals the number of matching genre tags under the stated rule.

10.6 Correctness of Ranking

The comparator places a movie with a higher score before a movie with a lower score. For equal scores, it places the alphabetically earlier title first. After sorting, the catalogue therefore follows the defined ranking rule.

10.7 Performance Notes

The sample contains only eight movies, so execution is very fast. Real recommendation services handle far more items and user interactions, and may use candidate generation, learned models, distributed processing, experiments and other techniques beyond this classroom example.

CHAPTER 11: LIMITATIONS, FUTURE SCOPE AND CONCLUSION

11.1 Limitations

- The catalogue and preference profile are hard-coded.
- Only genre overlap is used; no real user ratings or viewing history are processed.
- All preferred genres have equal weight.
- The program does not learn from user feedback.
- The score is not a probability and does not guarantee that a user will like a title.
- The example does not reproduce Netflix's private production algorithms.

11.2 Future Enhancements

- Allow users to enter preferred genres and a top-k value.
- Load movie details from a CSV file or database.
- Add weighted genre preferences and more movie metadata.
- Create a rating matrix and demonstrate a small collaborative-filtering example.
- Combine content-based and collaborative scores into a hybrid ranking.
- Evaluate recommendations using precision, recall or ranking metrics on a suitable test dataset.
- Build a graphical interface while protecting user data and providing clear controls.

11.3 Conclusion

This project explains how recommendation algorithms help streaming services organise large catalogues and suggest potentially relevant movies. Content-based filtering uses information about items and a viewer's preferences, collaborative filtering uses patterns across users or items, and hybrid systems combine multiple signals. Ranking algorithms arrange candidate titles so that the most relevant estimates appear first.

The C++ demonstration applies a simple and explainable content-based method. It counts genre matches, sorts movies by score and prints the resulting list. Its small scale makes it suitable for learning data structures, loops, functions, sorting and complexity analysis. The central lesson is that recommendations depend on both the data and the scoring rule, and a simplified classroom model should not be confused with Netflix's actual proprietary system.

REFERENCES AND APPENDIX

References

1. Netflix Technology Blog. Articles explaining recommendation systems and personalisation. https://netflixtechblog.com/

2. Netflix Help Center. Information about recommendations and how Netflix personalises the viewing experience. https://help.netflix.com/

3. Ricci, F., Rokach, L., and Shapira, B. (eds.). Recommender Systems Handbook. Springer.

4. Burke, R. (2002). Hybrid Recommender Systems: Survey and Experiments. User Modeling and User-Adapted Interaction, 12, 331–370.

5. Herlocker, J. L., Konstan, J. A., Terveen, L. G., and Riedl, J. T. (2004). Evaluating Collaborative Filtering Recommender Systems. ACM Transactions on Information Systems, 22(1), 5–53.

6. cppreference.com. C++ standard library reference for vector, algorithm, sort and string. https://en.cppreference.com/.